Guía completa del ambiente iShop

Tienda, chatbot RAG, n8n, Chatwoot, Qdrant, Ollama y Gemini

Proyecto de tesis iShop

24 de septiembre de 2026

Índice

1. Objetivo de esta guía 2
2. Qué se cambió en la versión final 2
   Estado verificado 3
3. Programas necesarios 3
4. Actualizar el clon que Paul ya tiene 3
   Paso 1: abrir Terminal 3
   Paso 2: entrar en la carpeta del proyecto 3
   Paso 3: revisar que está en la carpeta correcta 4
   Paso 4: detener el ambiente anterior 4
   Paso 5: descargar la versión nueva 4
   Paso 6: conservar el archivo privado 4
5. Configurar las variables privadas 4
6. Levantar todo el ambiente 4
   Direcciones 5
   Levantar la tienda estática 5
7. Configuración inicial de Chatwoot 5
   Crear el propietario 5
   Obtener el Access Token 5
   Crear el inbox, widget y AgentBot 5
8. Importar los workflows de n8n 6
9. Crear las credenciales en n8n 6
   Ollama API 6
   Qdrant API 6
   Google Gemini 6
10. Publicar el workflow principal 6
11. Cargar y actualizar el RAG 7
   Actualizar los documentos desde la tienda 7
   Crear nuevamente el índice 7
   Verificar visualmente 7
12. Cómo funciona Gemini cuando no hay créditos 7
13. Uso diario 8
   Para iniciar 8
   Para detener 8
14. Chat humano 8
15. Qué modificar y dónde 9

Cambiar productos y precios ... 9
Cambiar políticas, pagos, garantía o información pública ... 9
Cambiar reglas generales del asistente ... 9
Cambiar colores o texto del chat ... 9
16. Ejecutar las pruebas automáticas ... 9
17. Prueba manual recomendada ... 10
18. Problemas frecuentes ... 10
Docker no responde ... 10
Chatwoot abre pero el bot no responde ... 10
Error 404 del webhook ... 11
Productos incorrectos o antiguos ... 11
Gemini devuelve 429 ... 11
El widget no aparece ... 11
Un puerto está ocupado ... 11
19. Archivos importantes ... 11
20. Lista final antes de una demostración ... 11
21. Soporte y seguridad ... 12

1. Objetivo de esta guía

Esta guía permite que Paul levante y administre el proyecto aunque no sea desarrollador.

El proyecto incluye:

- La copia local de la tienda iShop.
- Un chat web de Chatwoot.
- Un asesor virtual coordinado por n8n.
- Una base de conocimiento RAG almacenada en Qdrant.
- Embeddings locales generados por Ollama.
- Gemini para comprender preguntas y redactar respuestas naturales.
- Traspaso de la conversación a una persona.

No se necesita modificar código para el uso normal. Los comandos de esta guía se copian y pegan en la aplicación **Terminal** de macOS.

2. Qué se cambió en la versión final

El workflow principal es:

iShop 04 - AgentBot Chatwoot

Ahora el recorrido RAG aparece con nodos separados en n8n:

```
Mensaje de Chatwoot
  → Gemini entiende la petición
  → Ollama crea el embedding
  → Qdrant busca evidencia en ishop_rag_v2
  → n8n ordena productos, precios y presupuesto
  → Gemini redacta usando únicamente la evidencia
  → Chatwoot muestra la respuesta
```

Si Gemini no tiene cuota:

```
Mensaje de Chatwoot
  → interpretación segura de respaldo
  → Ollama crea el embedding
  → Qdrant busca evidencia
  → n8n genera una respuesta verificada
```

Esto explica por qué el sistema puede seguir respondiendo sin créditos. Sin embargo, las respuestas son más naturales y conversacionales cuando Gemini está disponible.

Estado verificado

- Qdrant: colección `ishop_rag_v2` activa.
- Documentos indexados: 219.
- Un vector independiente por producto o documento informativo.
- Workflow 04: 22 nodos.
- Pruebas posteriores a las correcciones: 39 de 39 respuestas correctas en tres ejecuciones completas.
- Tiempo promedio observado sin cuota de Gemini: entre 2,15 y 4,70 segundos.
- Handoff humano: conversación abierta y asignada al asesor.

3. Programas necesarios

Paul debe tener instalados:

1. **Docker Desktop para Mac**
   - Descargar desde https://www.docker.com/products/docker-desktop/
   - Abrir Docker Desktop antes de iniciar el proyecto.
2. **Git**
   - macOS normalmente lo instala al ejecutar `git` por primera vez.
3. **Python 3**
   - Comprobar en Terminal:

```
python3 --version
```

4. Un navegador, por ejemplo Chrome.
5. Una API key de Gemini:
   - Crear o revisar en https://aistudio.google.com/app/apikey
   - Para máxima calidad y continuidad se recomienda activar facturación en el proyecto de Google asociado a la API key.

4. Actualizar el clon que Paul ya tiene

Paso 1: abrir Terminal

Abrir **Terminal** desde Aplicaciones.

Paso 2: entrar en la carpeta del proyecto

Cambiar la ruta si Paul clonó el proyecto en otro lugar:

```
cd ~/Documents/ishop
```

Paso 3: revisar que está en la carpeta correcta

```
git status
```

Debe aparecer el repositorio `ishop-tesis`.

Si Git muestra archivos modificados por Paul, no debe borrarlos. Debe enviar una captura a Pierre antes de continuar.

Paso 4: detener el ambiente anterior

```
bash chatbot/stop.sh
```

Este comando conserva las bases de datos y conversaciones. No elimina los volúmenes.

Paso 5: descargar la versión nueva

```
git pull origin main
```

Paso 6: conservar el archivo privado

El archivo `chatbot/.env` no se descarga desde Git porque contiene claves. Si ya existe, se conserva.

Nunca enviar por WhatsApp, correo o capturas:

- `GEMINI_API_KEY`
- `CHATWOOT_ACCESS_TOKEN`
- `CHATWOOT_BOT_TOKEN`
- `CHATWOOT_SECRET_KEY_BASE`

5. Configurar las variables privadas

Abrir el archivo:

```
chatbot/.env
```

Debe mantener esta estructura:

```
GENERIC_TIMEZONE=America/Guayaquil
CHATWOOT_SECRET_KEY_BASE=valor_generado
CHATWOOT_POSTGRES_PASSWORD=ishop_chatwoot_local
CHATWOOT_ACCOUNT_ID=numero_de_cuenta
CHATWOOT_ACCESS_TOKEN=token_de_chatwoot
CHATWOOT_BOT_TOKEN=token_del_agentbot
GEMINI_API_KEY=clave_de_google_ai_studio
```

No copiar los textos `valor_generado`, `numero_de_cuenta` ni `token_de_chatwoot`. Deben usarse los valores reales de la computadora de Paul.

Si `.env` no existe:

```
cp chatbot/.env.example chatbot/.env
```

El script de inicio generará automáticamente el secreto de Chatwoot.

6. Levantar todo el ambiente

Desde la raíz del proyecto:

```
bash chatbot/start.sh
```

El comando levanta:

- n8n
- Chatwoot
- PostgreSQL
- Redis
- Qdrant
- Ollama

Esperar hasta que Terminal muestre **Servicios iniciados**.

Direcciones

- Tienda: http://localhost:8080
- Chatwoot: http://localhost:3000
- n8n: http://localhost:5678
- Qdrant: http://localhost:6333/dashboard
- Ollama: http://localhost:11434

Levantar la tienda estática

Abrir una segunda ventana de Terminal:

```
cd ~/Documents/ishop
python3 -m http.server 8080
```

Esa Terminal debe permanecer abierta mientras se usa la tienda.

7. Configuración inicial de Chatwoot

Esta sección solo es necesaria la primera vez o si se perdió la configuración.

Crear el propietario

1. Abrir http://localhost:3000
2. Crear el usuario propietario.
3. Iniciar sesión.

Obtener el Access Token

1. Entrar a **Profile Settings**.
2. Buscar **Access Token**.
3. Copiarlo.
4. Pegar el valor en `chatbot/.env`:

```
CHATWOOT_ACCESS_TOKEN=valor_copiado
```

Crear el inbox, widget y AgentBot

```
python3 chatbot/scripts/setup_chatwoot.py
```

El script:

- Crea o reutiliza el inbox `iShop Web`.
- Crea el AgentBot `Asistente iShop`.
- Conecta Chatwoot con el webhook de n8n.
- Guarda el token público del widget.

- Guarda el token del bot en `.env`.

Después ejecutar:

```
docker compose -f chatbot/compose.yml restart n8n
```

8. Importar los workflows de n8n

Ejecutar:

```
bash chatbot/import-workflows.sh
```

Abrir http://localhost:5678 y revisar estos workflows:

1. `iShop 01 - Cargar conocimiento RAG`
2. `iShop 02 - Chatbot web con RAG`
3. `iShop 03 - Solicitud para asesor por email`
4. `iShop 04 - AgentBot Chatwoot`

El workflow principal es el número 04.

Si aparecen duplicados, dejar activo únicamente el workflow 04 más reciente. No deben existir dos workflows activos con el mismo webhook.

9. Crear las credenciales en n8n

En n8n abrir **Credentials**.

Ollama API

- Nombre: `Ollama account`
- URL: `http://ollama:11434`

Asignar esta credencial al nodo **Embeddings Ollama** del workflow 04.

Qdrant API

- Nombre: `Qdrant account`
- URL: `http://qdrant:6333`
- API key: dejar vacía. Si n8n obliga a escribir algo, usar `local`.

Asignar esta credencial al nodo **Buscar evidencia en Qdrant**.

Google Gemini

El workflow 04 utiliza `GEMINI_API_KEY` desde `chatbot/.env`.

Los nodos visibles son:

- `Gemini - Entender petición`
- `Gemini - Redactar con evidencia`

Después de cambiar la API key:

```
docker compose -f chatbot/compose.yml restart n8n
```

10. Publicar el workflow principal

En n8n:

1. Abrir `iShop 04 - AgentBot Chatwoot`.
2. Comprobar que no existen nodos rojos.
3. Guardar.
4. Pulsar **Publish** o **Activate**.

Si el workflow no está publicado, Chatwoot recibirá un error 404 y la IA no responderá.

11. Cargar y actualizar el RAG

La carga principal no depende del workflow 01. Se utilizan dos comandos controlados para evitar mezclar productos en fragmentos grandes.

Actualizar los documentos desde la tienda

```
python3 chatbot/scripts/sync_knowledge.py
```

Esto actualiza:

- `chatbot/knowledge/catalogo-productos.md`
- `chatbot/knowledge/contenido-publico-ishop.md`

Crear nuevamente el índice

Qdrant y Ollama deben estar levantados.

```
python3 chatbot/scripts/index_knowledge.py
```

El proceso:

1. Lee cada producto por separado.
2. Calcula el precio mínimo publicado.
3. Conserva URL, categoría, variantes y descripción.
4. Crea un embedding compacto con Ollama.
5. Recrea `ishop_rag_v2`.

Debe terminar con un mensaje parecido a:

```
Colección ishop_rag_v2: 219 documentos independientes
```

No cerrar Terminal mientras se está indexando.

Verificar visualmente

Abrir:

http://localhost:6333/dashboard

Buscar la colección:

```
ishop_rag_v2
```

12. Cómo funciona Gemini cuando no hay créditos

Sin créditos o con la cuota gratuita agotada, Google devuelve el error 429.

El flujo no se rompe:

1. Usa una interpretación local de respaldo.
2. Ollama sigue creando el embedding.
3. Qdrant sigue buscando evidencia.

4. n8n construye una respuesta segura.

En este modo las respuestas son rápidas, pero menos naturales.

Con créditos:

1. Gemini entiende contexto, intención y errores ortográficos.
2. Qdrant recupera evidencia.
3. Gemini mejora la redacción usando únicamente esa evidencia.

Si Gemini vuelve a fallar, el flujo conserva automáticamente la respuesta segura. Nunca debe responder sin consultar la base para una pregunta comercial.

13. Uso diario

Para iniciar

Terminal 1:

```
cd ~/Documents/ishop
bash chatbot/start.sh
```

Terminal 2:

```
cd ~/Documents/ishop
python3 -m http.server 8080
```

Abrir http://localhost:8080.

Para detener

Detener primero el servidor de la tienda con `Control + C`.

Después:

```
bash chatbot/stop.sh
```

Los datos se conservan.

Nunca ejecutar:

```
docker compose down -v
```

La opción `-v` elimina volúmenes y puede borrar conversaciones, configuración y bases locales.

14. Chat humano

La IA responde cuando una conversación está en estado `pending`.

Si el cliente escribe “quiero hablar con un asesor”:

1. La conversación cambia a `open`.
2. Se asigna a una persona.
3. La IA deja de contestar.
4. El asesor responde desde Chatwoot.

Para devolver la conversación a la IA:

1. Cambiar el estado a `pending`.
2. Volver a asignar el AgentBot si fuera necesario.

15. Qué modificar y dónde

Cambiar productos y precios

No editar manualmente el workflow.

Ejecutar:

```
python3 chatbot/scripts/sync_knowledge.py
python3 chatbot/scripts/index_knowledge.py
```

Cambiar políticas, pagos, garantía o información pública

Editar:

`chatbot/knowledge/contenido-publico-ishop.md`

Después:

```
python3 chatbot/scripts/index_knowledge.py
```

Cambiar reglas generales del asistente

Documento de referencia:

`chatbot/knowledge/reglas-asistente.md`

En el workflow 04, la interpretación se configura en:

`Preparar petición Gemini`

La redacción final se configura en:

`Preparar redacción final`

Paul no debería editar esos nodos sin guardar antes una copia del workflow.

Cambiar colores o texto del chat

Revisar:

```
chatbot/widget/
chatbot/scripts/setup_chatwoot.py
```

Después volver a ejecutar:

```
python3 chatbot/scripts/setup_chatwoot.py
```

16. Ejecutar las pruebas automáticas

Con todos los servicios levantados:

```
python3 chatbot/scripts/test_chatbot_e2e.py
```

La prueba revisa:

- Saludo rápido.
- MacBook para programación y presupuesto.
- Comparación entre opciones.
- Error ortográfico `microfno`.
- Error ortográfico `relok`.
- Envíos.

- Tarjetas y pagos.
- Garantía.
- Servicio técnico.
- iPhone.
- Audífonos.
- Mochila compatible con MacBook.
- Traspaso a un asesor.

El resultado esperado es:

```
Resultado: 13/13 correctas
```

La prueba crea una conversación llamada `Prueba automática RAG` en Chatwoot. Se puede archivar después.

17. Prueba manual recomendada

Escribir en este orden:

1. `Hola`
2. `Busco una laptop para programar, tengo 2000`
3. `Compara la opción 1 y la 2`
4. `Ahora quiero un microfno`
5. `Mejor un relok`
6. `¿Cuánto cuesta el envío a domicilio?`
7. `¿Qué tarjetas aceptan?`
8. `Quiero hablar con un asesor`

Comprobar:

- No aparecen adaptadores cuando se pide una laptop.
- La comparación usa exactamente las opciones mostradas.
- Micrófono no se confunde con Bose SoundLink Micro.
- Reloj devuelve Apple Watch.
- El envío indica $6 o gratuidad sobre $100.
- El handoff deja la conversación `open`.

18. Problemas frecuentes

Docker no responde

Abrir Docker Desktop y esperar a que indique que está ejecutándose.

Después:

```
bash chatbot/start.sh
```

Chatwoot abre pero el bot no responde

1. Comprobar que el workflow 04 está publicado.
2. Ejecutar:

```
python3 chatbot/scripts/setup_chatwoot.py
docker compose -f chatbot/compose.yml restart n8n
```

Error 404 del webhook

Publicar nuevamente el workflow 04 y reiniciar n8n.

Productos incorrectos o antiguos

```
python3 chatbot/scripts/sync_knowledge.py
python3 chatbot/scripts/index_knowledge.py
```

Gemini devuelve 429

La cuota está agotada. El respaldo seguirá funcionando.

Para recuperar respuestas más naturales:

1. Revisar facturación en Google AI Studio o Google Cloud.
2. Confirmar que la API key pertenece al proyecto con facturación.
3. Reiniciar n8n.

El widget no aparece

1. Confirmar que la tienda se abrió desde `http://localhost:8080`.
2. Ejecutar `setup_chatwoot.py`.
3. Recargar el navegador sin caché.

Un puerto está ocupado

Cerrar otras instalaciones que utilicen:

- 3000
- 5678
- 6333
- 8080
- 11434

19. Archivos importantes

```
chatbot/compose.yml
chatbot/start.sh
chatbot/stop.sh
chatbot/import-workflows.sh
chatbot/.env.example
chatbot/workflows/04-chatwoot-agent-bot.json
chatbot/scripts/setup_chatwoot.py
chatbot/scripts/sync_knowledge.py
chatbot/scripts/index_knowledge.py
chatbot/scripts/test_chatbot_e2e.py
chatbot/knowledge/catalogo-productos.md
chatbot/knowledge/contenido-publico-ishop.md
chatbot/knowledge/reglas-asistente.md
```

20. Lista final antes de una demostración

- Docker Desktop abierto.
- `bash chatbot/start.sh` ejecutado.
- Servidor de tienda en el puerto 8080.
- Workflow 04 publicado.

- Chatwoot muestra al asesor como disponible.
- Qdrant contiene `ishop_rag_v2`.
- `.env` contiene los tokens sin espacios adicionales.
- Prueba automática 13/13.
- Prueba de asesor humano completada.
- Ninguna clave privada aparece en capturas o documentos.

21. Soporte y seguridad

Si algo falla, Paul debe enviar:

1. Captura del error.
2. El nombre del nodo rojo en n8n.
3. El comando ejecutado.
4. El resultado de:

```
docker compose -f chatbot/compose.yml ps
```

No debe enviar el contenido de `chatbot/.env`.